Como los problemas pueden conllevar distintas soluciones (distintos algoritmos) debemos disponer de alguna forma de compararlos y determinar cuál es el mejor en distintos aspectos. Para esto usamos la *complejidad algorítmica* que es una *métrica lógica*.

Características de los Algoritmos.

- **Eficaces:** cumplen con el requerimiento solicitado.
- **Eficientes:** lo hacen mejor que otros.
- Pueden ser *legibles, claros y bien estructurados*.
- *Generales* (capaces de resolver una clase de problemas lo más amplia posible), de *uso y mantenimiento fácil*.
- **Eficiencia:** capacidad del algoritmo de resolver el problema usando la menor cantidad de espacio en memoria posible {*eficiencia espacial*} o alcanzandola en el tiempo más corto {*eficiencia temporal*}.
    - *Complejidad temporal:* tiempo que consume para resolver el problema.
    - *Complejidad espacial:* espacio que utiliza.
    - En general $complejidad\ temporal\ >\ complejidad\ espacial$ en términos de importancia.

Complejidad (temporal).

- Depende de:
    - Técnica utilizada (algoritmo)
    - Tamaño de la entrada
    - Hardware y Software
- Cómo medirlo
    - Experimentalmente → "a posteriori"
    - Estimándolo matemáticamente → "a priori"

Eficiencia de los algoritmos.

- Se distinguen 3 casos:
    - **Mejor caso:** mínimo tiempo posible.
    - **Caso medio:** tiempo promedio.
    - **Peor caso:** máximo tiempo posible. Es el caso de mayor interés.

Cálculo de la expresión T(N): Se realiza un *análisis matemático del algoritmo*, ya sea para calcular una expresión exacta de T(N) o una cota superior de la misma. T(N) es una función que contiene el *número de elementos que deben ser procesados*.

- Cuando calculamos T(N) tenemos en cuenta el *peor caso*.
- Permite evaluar el tiempo de ejecución de un algoritmo independientemente del entorno del hardware y software.
- Interesa la *velocidad de crecimiento* del tiempo *respecto del tamaño de entrada*.
- Para la descripción del algoritmo se emplea un *pseudocódigo*, el cual tiene menos nivel de detalle que un programa.
- Los *bucles* son el término dominante para determinar la eficiencia.

Operaciones primitivas: Se contabilizan como 1 operación elemental.

- Asignación
- Llamada a método
- Operación aritmética, lógica o relacional
- Indexación en array
- Seguir una referencia
- Volver de un método

Complejidad - Función de orden: Se prescinde de las constantes multiplicativas y de los órdenes de magnitud menores, por ej.

$$T(N) = 3n^2 + 4n + 4 \rightarrow T(N) = o(n^2)$$

Notación O grande:

- Una *cota superior asintótica* es una función que sirve de cota superior de otra función cuando el argumento tiende a infinito.
- Se usa la notación de Landau:
  - O(g(x))
  - Orden de g(x)

  para referirse a *las funciones acotadas superiormente por la función g(x)*.
- La cota superior asintótica tiene gran importancia en la Teoría de la complejidad computacional ya que a partir de ahí se definen las *clases de complejidad*.
- Se acostumbra a escribir $f(x) = O(g(x))$

Principales órdenes de complejidad ordenados

Notación O	Orden
O(1)	Constante
O(log n)	Logarítmico
O(n)	Lineal
O(n^2)	Cuadrático
O(n^a)	Polinomial (a>2)
O(a^n)	Exponencial (a>1)
O(n!)	Factorial